ThriveLens: Food-R1 Adaptation for Human-Confirmed Nutrition and Offline-First Fitness Tracking

Thabang Masoabi (23BAI70376), Tinotenda Muzembe (23BAI70589), David Seola (23BAI70652),
Chisomo Blessings Moleni (23BAI70696), Michael Bulis (23BAI70620)
Bachelor of Engineering — Artificial Intelligence and Machine Learning

Abstract—ThriveLens integrates food recognition, nutrition logging, exercise tracking, planning, and contextual coaching within an offline-capable wellness application. This paper describes the implemented application and reports the delivered Food-R1 fine-tuning experiment. The completed run used supervised, four-bit quantized low-rank adaptation on two NVIDIA T4 GPUs, with rank-16 adapters and a capped training set of 24,560 records. One epoch completed at 1,535 global steps. The trainer-reported mean training loss was 0.166157, the last logged training loss was 0.077358, and the single end-of-epoch validation loss was 0.149891. Greedy evaluation on 400 held-out records produced 79.25 percent exact match, 86.63 percent token F1, and 80.00 percent JSON validity on structured tasks. These aggregate results are dominated by produce classification; nutritional tasks have small sample counts and substantial numeric errors. The application therefore separates model-generated food candidates from user-confirmed portions and catalogue-derived nutrient calculations. Its local database and synchronization outbox support record persistence, while model inference remains server-side. The study establishes an implemented integration and a completed, bounded adaptation experiment, not clinical accuracy or full-test-set generalization. Training and validation illustrations are plotted from the saved trainer history.

Keywords—ThriveLens, Food-R1, QLoRA, food recognition, human confirmation, offline-first fitness tracking.

I. Introduction

ThriveLens is a team-developed wellness application that brings meal logging, workout execution, progress records, planning, food discovery, and coaching into one workflow. Its practical aim is to reduce the separation between identifying food, recording a meal, organizing exercise, and reviewing personal history. The final repository contains the mobile client, inference service, training notebooks, dataset-building scripts, and project documentation. This paper describes that repository at commit adaea5f437812344e8192ec193f45062 e5834ccc, dated 23 September 2026 [1]. The project is available at:

https://github.com/masterkush808/thrivelens-2026

A central design decision is that recognition and logging are different operations. A food model proposes an observation; the user confirms or corrects the food and specifies a portion; a nutrition lookup supplies a per-100-g basis; and deterministic arithmetic produces the quantities available for logging. The interface can use the resulting record as context for later coaching. A prediction is therefore not, by itself, an authorized database write or a verified nutritional measurement.

The machine-learning component adapts the public Food-R1 checkpoint for the project's food tasks. The delivered archive contains a completed run, saved adapters, training history, and a held-out evaluation [2]. It supports reporting actual training and generation results, but its scope must be stated accurately: the dataset build contains 302,157 supervised records, the completed run trains on a 24,560-record cap, and generation is evaluated on 400 records. These are three different quantities, not interchangeable descriptions of the experiment.

The contribution is an engineering integration accompanied by a resource-constrained adaptation study. The application joins persistent personal records to server-side multimodal inference and separates food recognition from nutrient arithmetic. The experiment documents the configuration that actually ran, the observed loss history, and task-specific output quality. It does not introduce a new optimizer, a custom composite loss, or a clinical assessment method. This distinction is essential because strong label-generation results do not establish reliable portion estimation or macronutrient prediction.

The paper addresses two questions. First, what Food-R1 adaptation and held-out performance are supported by the delivered artifacts? Second, how does the implemented application use model output without treating its generated nutrition values as the source for confirmed meal quantities? The first is answered through the saved run and evaluator reports; the second through the final application and serving code. Claims of improved health outcomes, deployment reliability, and performance gains over the unadapted checkpoint are outside the available evidence.

II. Technical Context

Food-R1 is the public checkpoint loaded by the training and serving code [3]. The delivered configuration identifies its implementation as Qwen3VLForConditionalGeneration. This experiment uses supervised fine-tuning; the name of the base model is not evidence that reinforcement learning was performed in this ThriveLens run.

Low-Rank Adaptation (LoRA) trains low-rank parameter updates while retaining a frozen pretrained backbone [4]. QLoRA extends this approach to a quantized backbone and introduces NormalFloat four-bit representation, double

quantization, and paged optimization for memory-efficient adaptation [5]. ThriveLens uses these established mechanisms rather than proposing a new adaptation algorithm. DeepSpeed ZeRO partitions training state to reduce redundant memory use across data-parallel workers [6]; the delivered run selects stage 2 with two GPUs.

The project's three data sources have different roles. Open Images supplies food-crop classification examples [7], Fruits-360 supplies produce-variety recognition examples [8], and Nutrition5k provides dish-level ingredient and nutritional supervision, including depth information [9]. These sources should not be interpreted as interchangeable meal photographs. Classification targets and structured nutritional targets require different evaluation criteria, and their relative frequencies affect an aggregate score. The project-specific counts and task expansion are established by its build-verification file, not by the original publications' dataset sizes.

III. Application Architecture and Implemented Arithmetic

A. Mobile records and service boundaries

The repository separates an Expo/React Native mobile client from Python serving code. The client uses Expo Router, a SQLite database through expo-sqlite, and Drizzle data access. Supabase is the cloud data layer. The application also includes OpenStreetMap/Overpass place discovery, Open Food Facts barcode integration, workout records, meal plans, fridge inventory, schedules, and progress-related views. The implementation described in this section is drawn from the final source tree [1]; the existence of code is not treated as a measured native-device acceptance result.

The local schema contains food logs, exercises, workout plans and sessions, workout logs, progress logs, notifications, fridge items, meal plans, and an outbox. Food records include fields for portion mass, energy and macronutrients, image reference, model version, model confidence, and nutrient source. These fields permit the interface to retain the distinction between an AI observation and a source-labelled meal entry. Food-log repository functions include user-filtered queries, while the outbox identifies pending operations by user and operation identifier.

The outbox implementation supports upsert/delete operations, retry attempts, next-attempt timestamps, base versions, deletion timestamps, and a conflict status. Enqueuing an existing operation identifier is ignored, and failed operations receive an increasing retry delay capped at 60 seconds. These are concrete persistence mechanisms, not evidence that every write path has been independently proven atomic or that synchronization has zero conflicts. Offline-first describes the local record layer; Food-R1 weights are not loaded by the mobile app.

The serving layer provides food analysis, nutrition lookup, and coach-chat interfaces. A FastAPI service exposes these operations, and a Hugging Face ZeroGPU/Gradio entry point reuses the model loader and catalogue logic. The client selects configured service routes and handles upload, timeout, and response errors. Image recognition and generative coaching therefore require an available inference service. A successful offline database read does not demonstrate offline neural inference.

B. Food candidates and confirmed nutrient quantities

The camera-serving path invokes food_classification and ingredient_recognition. It does not invoke the Nutrition5k nutrition_estimation or full_food_analysis tasks on a plain mobile photograph. Parsed outputs are returned as candidates with a model revision and confidence information; low-scoring or invalid outputs can produce abstention. The loader's confidence is the arithmetic mean of the maximum softmax probability at each generated token. This is an implemented generation statistic, not an empirically calibrated probability that the food identification is correct.

After food and portion confirmation, the lookup uses a static, curated per-100-g catalogue and, when enabled, the Open Food Facts provider. An unmatched lookup returns an abstained result instead of a confirmed nutrient object. The static catalogue labels its values as approximate and uses the generic source reference usda-fdc-curated. That source label should not be mistaken for an individually verified FoodData Central accession for every entry.

The actual portion computation appears in serving/app/main.py and serving/space_zero/app.py. For a confirmed amount m in grams and a catalogue value b_j per 100 g for nutrient j, the implementation computes:

$$n_j(m) = \mathrm{round}_1\left(\frac{b_j\, m}{100}\right) \quad (1)$$

Here, round_1 means the Python round(value, 1) operation used by round_one. The same scaling is applied separately to energy, protein, fat, carbohydrate, and available fibre; missing fibre remains null. Energy is expressed in kilocalories and the macronutrient quantities in grams. Equation (1) is a transcription of the implemented lookup, not an image-to-calorie estimation model or a newly proposed nutritional formula.

This separation has an important limitation as well as a benefit. It prevents the application's confirmed nutrient object from being populated directly with the food model's generated numeric answer. It does not prove that the user-selected food is correct, that a stated portion was weighed, or that a catalogue entry exactly matches a preparation. Determinism establishes repeatable arithmetic for supplied inputs, not the accuracy of those inputs.

C. Contextual coaching and action control

The coach context includes profile information, recent food records, fridge items, meal and workout plans, schedules, adherence fields, and permission-aware location context. The action registry supports meal logging, place discovery, navigation, workout scheduling, meal planning and preparation, fridge additions, goal changes, grocery suggestions, and the project's cheat-day workflow. The client

validates supported action types, expected keys, and relevant numeric constraints before accepting a proposed action.

Execution states distinguish suggestions, approval requests, execution, completion, failure, and cancellation. The serving code also contains refusal logic for medical or unsafe requests and returns context references alongside replies. These controls describe the application's intended authority boundary: generated language may propose an action, but it is not itself proof of successful execution. The archive does not contain a user study of coaching quality or a deployed-system safety evaluation, and this paper does not assign such outcomes to these mechanisms.

IV. Dataset and Completed Training Configuration

A. Dataset units and partition sizes

Table I transcribes the task counts in ThriveLens_300K_BUILD_VERIFICATION.json. The full build contains 242,756 training records, 6,460 validation records, and 52,941 test records. Each Nutrition5k dish contributes three supervised tasks: ingredient recognition, nutrition estimation, and full food analysis. Thus, 3,490 dishes contribute 10,470 records, not 3,490 records. Adding these to 107,794 Open Images food-crop records and 183,893 Fruits-360 records gives exactly 302,157 supervised records. This total counts supervised task instances, rather than unique dishes or images.

TABLE I
FULL DATASET BUILD: SUPERVISED RECORDS BY TASK

Task	Train	Validation	Test
Food classification	96,754	5,638	5,402
Produce variety	137,932	0	45,961
Ingredient recognition	2,690	274	526
Nutrition estimation	2,690	274	526
Full food analysis	2,690	274	526
Total	242,756	6,460	52,941

Source: repository build-verification JSON [1]. Each Nutrition5k dish produces one record for each of its three tasks. The executed training cap is 24,560 records, not the full training column.

The build-verification report also lists 298,667 referenced images and 234,080 leakage-safe groups. These are reported build properties, not a new image-level audit performed for this paper. The full image archive and capped training manifest are not included in the final adapter delivery, so their memberships cannot be independently reconstructed from that delivery alone.

The validation split contains no produce-variety records. It consists of food classification and the three Nutrition5k tasks. Consequently, its loss does not directly measure the task that dominates the held-out generation subset. Nor should the complete build be described as balanced across tasks, countries, or everyday mobile-camera conditions. The intended breadth of the application is a product goal, not a representativeness result established by these counts.

B. The executed training subset and parameters

The final command points to train_capped.json. The budget override sets TRAIN_ROW_CAP to 24,560, and full_training.log confirms 24,560 examples, one epoch, an effective batch size of 16, and 1,535 total optimization steps. The cap's task composition is not supplied; this paper does not infer that it preserves the full training partition's proportions. Table II reports the final configuration corroborated by the command, manifest, and saved adapter.

TABLE II
CONFIGURATION OF THE COMPLETED RUN

Parameter	Executed value
Base checkpoint	zy12123/Food-R1
Training population	24,560 capped records
Hardware / distributed	2 NVIDIA T4 GPUs; ZeRO-2
Epochs / global steps	1.0 / 1,535
Quantization / compute	4-bit NF4; double quant.; FP16
LoRA rank / alpha / dropout	16 / 16 / 0.05
Microbatch / accumulation	1 per GPU / 8 steps
Effective batch	16 records
Optimizer	Paged AdamW, 8-bit
Learning rate / schedule	0.0002 / cosine
Warm-up / weight decay	0.05 / 0.01
Gradient-norm limit	0.3
Maximum sequence length	192 tokens
Image pixel range	4,096–25,600 pixels
Validation / early stopping	End of epoch / disabled
Seed / deterministic training	2026 / disabled

Source: executed command, final manifest, budget override, and saved adapter [2]. Vision tower and merger are frozen. Input limits are pixel counts, not edge dimensions.

The vision tower and merger remain frozen. Language-side LoRA targets the eligible query, key, value, output, gate, up, and down linear projections, excluding name spans for visual components, embeddings, and the language-model head. The saved adapter contains 43,646,976 tensor elements in 504 tensors and occupies 83.32 MiB. These are adapter statistics; they do not represent the size of a standalone model, because inference also requires the base checkpoint.

The final command enables paged eight-bit AdamW, cosine learning-rate decay, a five-percent warm-up, weight decay, gradient clipping, and gradient checkpointing. Checkpoints are scheduled every 250 steps with two retained checkpoints. Early stopping and loading the best checkpoint at the end are disabled. The result is a fixed one-epoch supervised adaptation.

Several delivery files describe earlier settings. In particular, requested_configuration.json and verified_training_argument_contract.json retain rank-64 or larger input settings, whereas the executed command, budget override, final manifest, and saved adapter agree on rank 16 and the reduced input limits. Those final artifacts determine the configuration reported here. The image limits of 4,096 and 25,600 are pixel

counts; they must not be relabelled as the earlier image-size settings.

C. Runtime and experiment identity

The recorded environment is Python 3.12.13, PyTorch 2.10.0+cu128, Transformers 5.3.0, Accelerate 1.13.0, PEFT 0.15.2, bitsandbytes 0.49.2, DeepSpeed 0.17.5, and TRL 0.25.0. The manifest identifies trainer commit 70c7b2fcb0e276b1fa4b136852e9b862ce8730fa and completion on 15 September 2026. Seed and data seed are 2026; full deterministic training is disabled. Greedy evaluation, discussed next, is therefore distinct from a claim of bitwise-reproducible training.

The trainer reports 14,539.33 seconds for the run, approximately 4.04 elapsed hours. The budget file's projected GPU-hours are not substituted for this measured runtime. Four FP16 overflow/skipped-step messages occur in the log, followed by continued training and completion. Accordingly, 1,535 is reported as the final global-step count, not as a verified count of successful parameter updates without numerical interruptions. Validation reports 1,259.49 seconds for 6,460 examples.

V. Evaluation Procedure and Implemented Metrics

A. Generation scope

The delivered evaluation command uses two workers, the full test JSON as its input source, and --max-samples set to 400. Decoding is greedy, with do_sample=False, one beam, and a maximum of 256 generated tokens. It uses the final 4,096–25,600 pixel range and the quantized adapter path. The command, held-out report, task CSV, and metrics JSON define the evaluation reported below [2]. This is a capped held-out evaluation, not an evaluation of all 52,941 test records.

The evaluator separates execution failures, answer-tag compliance, normalized exact match, token overlap, structured parsing, ingredient sets, and numeric error. These quantities answer different questions. A request can execute successfully yet return the wrong food. An answer can be valid JSON while containing incorrect nutrition. An output can share many tokens with the reference but fail exact match or omit important ingredients. Reporting only the overall score would obscure these differences.

B. Text and ingredient scoring

The advanced evaluator embedded in the repository notebook extracts text between answer tags when present, otherwise uses the available output, and normalizes text by lowercasing and replacing non-alphanumeric runs with spaces. Exact match compares the resulting answers. Token F1 uses token multiplicities, rather than just membership in a set. Let T_p and T_r be the normalized predicted and reference token lists, and let O be the sum of the minimum occurrence counts of each token in the two lists. The implemented precision, recall, and F1 are:

$$P = \frac{O}{|T_p|}, \quad R = \frac{O}{|T_r|},$$
$$F_1 = \frac{2PR}{\max(P+R, 10^{-12})} \tag{2}$$

The code returns one when both token lists are empty and zero when only one is empty. Per-example F1 values are averaged. Ingredient scoring instead extracts normalized ingredient names into sets, counts their intersection, divides by each set size with a denominator floor of one, and computes the harmonic mean with the same small denominator guard. Its per-example values are also averaged. These are the evaluator's scoring rules [1], not additional metrics introduced after seeing the results.

C. Structured and numeric outputs

For structured tasks, the parser accepts a decoded dictionary or list. The reported JSON-validity rate therefore describes successful parsing, not complete schema conformity, correct ingredient content, or nutritional plausibility. Numeric scoring is more restrictive: both answers must yield dictionaries, the relevant field must be convertible to a finite number in each, and missing or invalid pairs are skipped.

For field f, let V_f denote the included finite prediction–reference pairs, let n_f be their count, and let $\hat{y}_{if}$ and y_{if} denote the predicted and reference values. The evaluator's mean absolute error and root mean squared error are:

$$\mathrm{MAE}_f = \frac{1}{n_f}\sum_{i \in V_f}\left|\hat{y}_{if} - y_{if}\right| \tag{3}$$

$$\mathrm{RMSE}_f = \sqrt{\frac{1}{n_f}\sum_{i \in V_f}(\hat{y}_{if} - y_{if})^2} \tag{4}$$

Mean absolute percentage error uses only pairs in A_f, the subset of V_f whose reference magnitude exceeds 10 to the power of minus eight:

$$\mathrm{MAPE}_f = \frac{100}{|A_f|}\sum_{i \in A_f}\left|\frac{\hat{y}_{if} - y_{if}}{y_{if}}\right| \tag{5}$$

The evaluator computes these quantities for mass_g, total_calories, fat_g, carb_g, and protein_g. MAE and RMSE retain the field's unit, whereas MAPE is a percentage. Equations (3)–(5) directly express the sums and denominator rules in add_numeric_scores and the reporting code [1]. The sample counts associated with each numeric result are necessary: excluding an unparsable answer is not equivalent to assigning it zero error.

The study does not add calibration scores, confidence intervals, cross-validation results, or statistical significance tests absent from the reports. It also does not convert loss into recognition accuracy. The available artifacts support descriptive reporting of this run and its selected held-out subset, rather than a comparative claim about model improvement.

VI. Results

A. Training and validation observations

The saved history contains 154 training-loss observations. The first logged value is 1.832903 at global step 1. The last logged interval loss is 0.077358 at step 1,530, while the trainer's aggregate train_loss is 0.166157 at completion. These values summarize different intervals and should not be substituted for one another. Figure 1 plots the training-loss entries in trainer_state.json; the trajectory declines sharply initially and then continues at lower logged values.

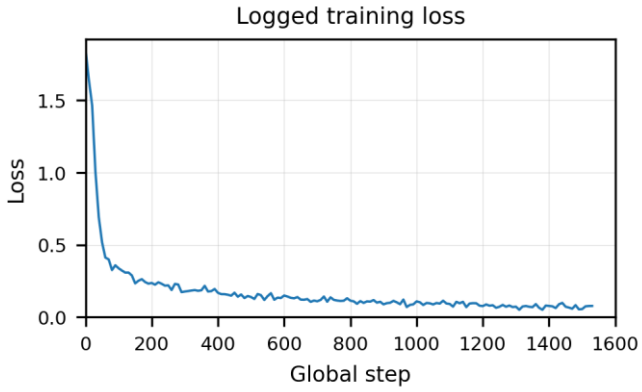


Fig. 1. Training loss plotted from the 154 saved history entries [2]. No smoothing or added observations are used; the last logged interval is at step 1,530.

There is exactly one validation measurement: 0.149891 at epoch 1.0 and step 1,535. Figure 2 shows this single point. No intermediate validation values are available, so a multi-epoch validation curve cannot be drawn from this experiment. The diagnostics file reports no final-versus-best regression because the only validation value is necessarily also the best observed value. This does not demonstrate absence of overfitting or establish the optimal epoch count.

The training and validation losses establish completion and describe the recorded optimization behavior. Their numerical difference is not a controlled generalization-gap estimate for a common task distribution: training uses a capped mixture, while validation omits produce-variety classification. The held-out generation results are therefore reported separately instead of treating low loss as sufficient evidence of application quality.

B. Held-out recognition and output structure

All 400 requested examples completed without a reported execution failure. Overall exact match is 79.25%, token F1 is 86.6320%, and answer-tag compliance is 99.25%. Structured-output parsing succeeds for 12 of 15 structured-task examples, giving 80.00% JSON validity. Table III provides the task counts and scores from the delivered report.

TABLE III
HELD-OUT GENERATION RESULTS

Task	n	EM %	F1 %	JSON %
Produce variety	344	82.85	89.21	—
Food classification	41	75.61	75.61	—
Ingredient recognition	3	33.33	81.33	100.00
Nutrition estimation	6	0.00	63.33	100.00
Full food analysis	6	0.00	40.04	50.00
Overall	400	79.25	86.63	80.00*

Source: held-out metrics [2]. EM: normalized exact match. F1: token F1. *Overall JSON validity uses only the 15 structured-task examples; a dash means not applicable.

Produce-variety classification contributes 344 of the 400 examples, or 86.00% of the subset. Its 82.85% exact match has a much larger influence on the overall score than the three ingredient-recognition examples. Food classification, the classification task used by the camera-serving path, has 41 examples and 75.61% exact match. The overall 79.25% figure must not be labelled universal food-recognition accuracy or nutritional accuracy for mobile photographs.

Ingredient-set precision, recall, and F1 are 0.5714, 0.4848, and 0.5185 for ingredient recognition. For full food analysis they are 0.4444, 0.2136, and 0.2446. These set scores differ from token F1 because repeated syntax and numeric tokens can contribute to text overlap without adding correct ingredient names. Full food analysis also returns answer tags and valid JSON in only three of its six examples.

C. Nutritional prediction errors

Table IV reports all five numeric fields for the two relevant tasks. Nutrition estimation contributes six valid pairs for each field; full food analysis contributes only three valid pairs from six attempted examples. The latter errors are consequently conditional on parseable outputs. They do not incorporate the three invalid outputs as a numeric penalty, and the two tasks' errors should not be compared without this qualification.

TABLE IV
NUMERIC ERRORS ON FINITE, PARSEABLE PAIRS

Field / task	MAE	RMSE	MAPE %
Nutrition estimation: 6/6 valid			
Mass (g)	59.17	93.55	56.69
Energy (kcal)	137.69	174.93	126.96
Fat (g)	7.38	10.83	67.34
Carbohydrate (g)	10.12	13.76	168.04
Protein (g)	6.07	7.68	122.25
Full analysis: 3/6 valid			
Mass (g)	63.33	90.35	39.96
Energy (kcal)	176.42	231.91	59.74
Fat (g)	10.13	12.59	64.26
Carbohydrate (g)	14.33	23.94	49.08
Protein (g)	13.40	15.65	71.02

Source: held-out metrics [2]. Valid/attempted counts apply to each field. MAE and RMSE use the field's unit. Invalid full-analysis outputs are excluded from numeric averages; they are not zero-error predictions.

For nutrition estimation, calorie MAE is 137.69 kcal and calorie MAPE is 126.96%. Carbohydrate and protein MAPE are 168.04% and 122.25%, respectively. Full food analysis has a calorie MAE of 176.42 kcal among its three included pairs. These small-sample observations do not establish reliable nutrient estimation. The lower MAPE of one task or field cannot compensate for missing outputs or justify a claim of clinical suitability.

The results support keeping the deployed application's catalogue calculation separate from the model's nutritional generation. This is a design interpretation of the observed errors, not an experiment demonstrating that human confirmation eliminates all error. No end-to-end comparison of confirmed meal logs against weighed, independently verified meals is contained in the delivery.

VII. DISCUSSION AND LIMITATIONS

A. What the experiment establishes

The experiment establishes that the selected Food-R1 checkpoint was adapted under the delivered dual-T4 configuration, that an adapter was saved, and that the bounded evaluation produced the reported label and structured-output scores. The strongest result is classification on this particular held-out mixture. The study does not establish the improvement attributable to fine-tuning, because there is no matched unadapted-model baseline, ablation, or repeated-seed comparison in the final reports.

The app's contribution is broader than the classifier but has a different evidence basis. Source inspection establishes interfaces, database structures, typed actions, lookup arithmetic, and model-loading paths. The training archive establishes model-run measurements. Neither alone establishes a production service-level guarantee, successful synchronization across physical devices, or sustained user adherence. The manuscript therefore separates implementation descriptions from experimentally observed model quality.

B. Dataset coverage and transfer to mobile use

The capped training population and the 400-record evaluation constrain generalization. The retained predictions file contains 200 examples, not every evaluated output; task totals can be checked against the saved aggregate files, but an independent reconstruction of every score is not possible from those retained predictions alone. Likewise, the original data images and the selected training-cap membership are not shipped inside the adapter archive.

Dataset modalities also matter. The project's Nutrition5k nutrition-estimation and full-analysis tasks expect paired RGB and depth inputs, while the camera-serving route uses a single RGB photograph and the classification/ingredient prompts. The nutritional task results therefore are not a validated mobile-photo calorie estimator. Controlled produce images, overhead meal images, and handheld photographs should not be assumed to have identical error behavior. The zero produce-variety count in validation further limits how its single loss value can be interpreted.

C. Serving and nutrition-source limitations

The repository's serving configuration and the held-out command are not an identical benchmark environment. In particular, the serving documentation specifies a 192-token generation limit, whereas the delivered held-out command allows 256. The model loader also has configurable precision and quantization. Reported held-out scores therefore identify the evaluation configuration, not a measured guarantee for every deployment setting or request path.

Confidence and abstention require similar caution. Averaging maximum token probabilities is not the same as validating food-class probability calibration, and the model card describes the abstention threshold as provisional. No result in the archive demonstrates reliable rejection of non-food photographs or confidence-threshold performance on live phone inputs. User correction and catalogue lookup are meaningful controls, but they cannot convert unsupported visual assumptions into measurements.

Catalogue provenance is also bounded. The curated static catalogue provides source-labelled approximate reference values, while provider lookups depend on availability and matching. Correct multiplication cannot repair an incorrect match, missing preparation detail, or an inaccurate portion. The application should therefore be understood as an informational tracking system, not a substitute for a food-composition measurement or clinical assessment. The delivered evaluation measures generated answers; it does not validate the entire nutrition-provider pipeline.

D. Reproducibility and availability

The primary software record is the linked repository [1]. The primary experimental record is ThriveLens_FoodR1_Dual_T4_QLoRA_Final.zip [2], particularly full_training_command.json, full_training.log, trainer_state.json, adapter_config.json, thrivelens_run_manifest.json, held_out_evaluation_command.json, and held_out_test_metrics.json.

The figures are plotted directly from the saved trainer history without smoothing or added observations. The original training_diagnostics.png provides a corresponding diagnostic record.

The delivered adapter is identified by "SHA-256 8a1c75a738100b9b43b31e972e9ae2e2ad5b23585c2a47c3 9db13bc76cf262c9". The manifest records extracted Kaggle dataset mode rather than a dataset-archive checksum; this is not a missing checksum to be filled with an unrelated file hash. The repository excludes large training archives and model weights, so its URL is the public software link, not a claim that all experimental data and weights are publicly redistributed there.

A fuller evaluation would require the exact training-cap manifest, complete held-out predictions, a matched base-model run, broader task coverage, and mobile end-to-end measurements. These are remaining evaluation needs, not completed experiments. The reported findings remain limited to the supplied final run and the pinned software snapshot.

VIII. Conclusion

ThriveLens combines offline-capable fitness and nutrition records with server-side Food-R1 inference, user confirmation, catalogue-based nutrient arithmetic, and contextual action-oriented coaching. The completed adaptation used 24,560 training examples, rank-16 NF4 QLoRA, two T4 GPUs, and one epoch ending at 1,535 global steps. The saved loss history and 400-record held-out evaluation provide concrete results, while the task breakdown reveals the dominance of produce classification and the limited reliability of generated nutritional quantities.

The defensible outcome is a delivered adaptation and an implemented separation between visual recognition and confirmed nutrition tracking. It is not proof of precise image-derived calories, broad mobile-domain generalization, or clinical benefit. Retaining that distinction preserves the application's vision while making the reported research consistent with its actual code, formulas, artifacts, and measured results.

Acknowledgment—The authors acknowledge supervision by Sant Kumar Maurya (E13548).

References

[1] ThriveLens Team, "ThriveLens 2026," software repository, commit adaea5f437812344e8192ec193f45062e5834ccc, Sep. 23, 2026. Includes application and serving code, training notebook, and dataset build-verification JSON. Accessed: Sep. 26, 2026. [Online]. Available: https://github.com/masterkush808/thrivelens-2026

[2] ThriveLens Team, "ThriveLens Food-R1 dual-T4 QLoRA final training delivery," ThriveLens_FoodR1_Dual_T4_QLoRA_Final.zip, experiment completed Sep. 15, 2026. Author-supplied experimental artifact: executed command, trainer history, adapter, diagnostics, and held-out reports. Not a separately published benchmark.

[3] zy12123, "Food-R1," Hugging Face model repository. Accessed: Sep. 26, 2026. [Online]. Available: https://huggingface.co/zy12123/Food-R1

[4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," arXiv:2106.09685, 2021. doi: 10.48550/arXiv.2106.09685. Online paper.

[5] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," arXiv:2305.14314, 2023. doi: 10.48550/arXiv.2305.14314. Online paper.

[6] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models," in Proc. SC, 2020. doi: 10.1109/SC41405.2020.00024. Online paper.

[7] A. Kuznetsova et al., "The Open Images Dataset V4: Unified Image Classification, Object Detection, and Visual Relationship Detection at Scale," arXiv:1811.00982, 2018. doi: 10.48550/arXiv.1811.00982. Online paper.

[8] H. Mureşan and M. Oltean, "Fruit Recognition from Images Using Deep Learning," arXiv:1712.00580, 2017. doi: 10.48550/arXiv.1712.00580. Online paper.

[9] Q. Thames, A. Karpur, W. Norris, F. Xia, L. Panait, T. Weyand, and J. Sim, "Nutrition5k: Towards Automatic Nutritional Understanding of Generic Food," in Proc. IEEE/CVF CVPR, 2021, pp. 8903–8911. [Online]. Available: CVPR 2021 open-access paper.